PRADITA University

IF140303-Web Application Development

Session-10:

GitHub OAuth: in Phoenix

PRU/SPMI/FR-BM-18/0222

Alfa Yohannis



Contents

PRADITA University

1 Introduction
2 How OAuth Works
3 Migration
4 Model
5 Dependencies
6 GitHub
7 Configuration
8 Environment Variables
9 View
10 Revoking Access / Logout
11 Router
12 Controller
13 Demo

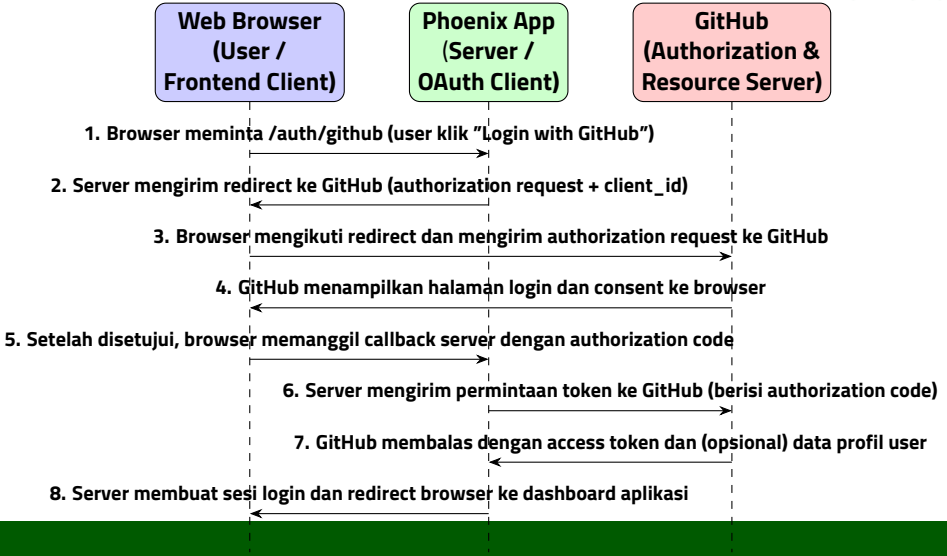
GitHub OAuth in Phoenix

PRADITA University

- OAuth lets applications verify user identity without handling passwords directly.
- Users grant permission to providers (e.g., GitHub) to share basic profile data securely.
- This reduces security risks and improves usability: users sign in without creating new accounts.
- In Phoenix, OAuth integration is simplified using the `Ueberauth` library.
- `ueberauth_github` provides “Login with GitHub” support following modern web best practices.

OAuth Authorization Code Flow

PRADITA University



How OAuth Works

PRADITA University

- OAuth 2.0 involves the browser, Phoenix server, and GitHub as authorization/resource server.
- The browser is redirected to GitHub for login and permission.
- GitHub returns an authorization code to Phoenix via callback.
- Phoenix securely exchanges the code for an access token using a server-to-server request.
- The token allows Phoenix to fetch the user's GitHub profile without ever handling passwords.

Creating the Users Table (Migration)

PRADITA University

The application requires a `users` table to store GitHub OAuth login data, including email, provider name, and access token. This migration file is placed inside `priv/repo/migrations/` and defines the necessary database structure.

```elixir
defmodule Hiwi.Repo.Migrations.AddUsers do
  use Ecto.Migration

  def change do
    create table(:users) do
      add :email, :string
      add :provider, :string
      add :token, :string
      timestamps()
    end
  end
end
```

```
mix ecto.migrate
== Running 20251102090628 Hiwi.Repo.Migrations.AddUsers.change/0 forward
== Migrated 20251102090628 in 0.03s
```

User Schema (Model)

PRADITA University

```elixir
defmodule Hiwi.User do
  use Ecto.Schema
  import Ecto.Changeset

  schema "users" do
    field :email, :string
    field :provider, :string
    field :token, :string
    timestamps(type: :utc_datetime)
  end

  def changeset(user, attrs) do
    user
    |> cast(attrs, [:email, :provider, :token])
    |> validate_required([:email, :provider, :token])
  end
end
```

Model Explanation

PRADITA University

- The `Hiwi.User` schema represents the `users` table created by the migration, defining the fields used to store GitHub OAuth login data.
- The schema includes three main fields: `email`, `provider`, and `token`, each mapped directly to database columns.
- The `timestamps/1` macro automatically generates `inserted_at` and `updated_at` columns with UTC datetime.
- The `changeset/2` function validates incoming data, casts only allowed fields, and ensures required values are present before inserting or updating records.
- This schema can be extended later with additional GitHub fields such as `name`, `avatar_url`, or `github_id` as the application grows.

Ueberauth Dependencies in mix.exs

PRADITA University

```elixir
defmodule Hiwi.MixProject do
  use Mix.Project

  def application do
    [
      mod: {Hiwi.Application, []},
      extra_applications: [ logger, :runtime_tools, :ueberauth
          , :ueberauth_github ]
    ]
  end

  defp deps do
    [
      # OAuth dependencies
      {:ueberauth, "~> 0.10"}, {:ueberauth_github, "~> 0.8.3"}
    ]
  end
end
```

```
mix deps.get
mix compile
```

Ueberauth: https://github.com/ueberauth/ueberauth

GitHub: https://github.com/ueberauth/ueberauth_github

Dependencies Explanation

PRADITA University

- Phoenix uses the `Ueberauth` ecosystem to support authentication through external providers such as GitHub, Google, or Facebook.
- The `ueberauth_github` strategy implements GitHub's OAuth 2.0 flow, handling authorization URLs, token exchange, and user profile retrieval.
- Both `:ueberauth` and `:ueberauth_github` must be added to the `deps` list and to the `extra_applications` list so they run as part of the OTP application.
- After updating `mix.exs`, running `mix deps.get` and `mix compile` downloads and builds the required libraries.
- With dependencies installed, the next step is configuring GitHub OAuth in `config/config.exs` and adding the routes `/auth/github` and `/auth/github/callback`.

Creating a GitHub OAuth App

PRADITA University

- Log in to your GitHub account.
- Open: `Settings` → `Developer Settings` → `OAuth Apps`
- Click **New OAuth App**.
- Fill in the required fields:
  - Application Name: `Hiwi`
  - Homepage URL: `https://www.hiwi.com`
  - Description: `Queue Management System`
  - Callback URL: `http://localhost:4000/auth/github/callback`
- Click **Register application**.
- Open the app again and click **Edit**.
- Copy the **Client ID**.
- Click **Generate a new client secret** and save it securely.

GitHub OAuth App Output

PRADITA University

- After registration you obtain:
  - **GITHUB_CLIENT_ID**
  - **GITHUB_CLIENT_SECRET**
- These values must be added as environment variables.
- They will be read by Ueberauth to authenticate securely.
- The Client Secret is shown only once — store it safely.

OAuth Configuration (config/config.exs)

```
1  import Config
2
3  config :ueberauth, Ueberauth.Strategy.Github.OAuth,
4    client_id: System.get_env("GITHUB_CLIENT_ID"),
5    client_secret: System.get_env("GITHUB_CLIENT_SECRET")
```

- Ueberauth reads GitHub OAuth credentials from environment variables.
- Storing secrets in the system environment prevents accidental exposure.
- Phoenix loads these values at runtime when handling OAuth requests.

Environment Variables for GitHub OAuth

```
$ nano ~/.bashrc

# GitHub OAuth
export GITHUB_CLIENT_ID="your_client_id"
export GITHUB_CLIENT_SECRET="your_client_secret"
```

```
$ source ~/.bashrc
```

- Environment variables store secrets safely outside source code.
- Loaded by the shell before Phoenix runs.
- Ueberauth reads these values at runtime for secure OAuth authentication.

View: Adding OAuth GitHub Button

PRADITA University

```
<!-- lib/hiwi_web/components/layouts.ex -->
<li>
  <a href="http://localhost:4000/auth/github"
     class="btn btn-ghost">
    OAuth GitHub
  </a>
</li>

<li>
  <a href="https://phoenixframework.org/"
     class="btn btn-ghost">
    Website
  </a>
</li>
```

View: How the OAuth Button Works

PRADITA University

- The button links to `/auth/github`, which starts the OAuth login flow.
- When clicked, Ueberauth redirects the browser to GitHub for authentication.
- CSS classes `btn btn-ghost` style the button (Tailwind/DaisyUI).
- This code lives in `layouts.ex`, making the button appear in the main navbar.
- After GitHub login, a callback to Phoenix completes the authentication.

Revoking Access / Logout

PRADITA University

OAuth logout is more than deleting a local session. GitHub may still hold valid access tokens, so users or administrators may need to revoke them for security reasons.

- **Revoke for an individual user**
  - GitHub → Settings → Applications → Authorized OAuth Apps
  - Select the app → click ⋮ → **Revoke**
  - Token becomes invalid; the user must log in again.
- **Revoke all user tokens (Admin-level)**
  - GitHub → Developer Settings → OAuth Apps
  - Select app → **Edit** → Revoke All User Tokens
  - All issued tokens become invalid globally.
- **Effects on Phoenix App**
  - API calls with old tokens fail (Unauthorized).
  - Phoenix can prompt re-login.
  - Local session may still exist, but GitHub access is blocked.

Router: OAuth GitHub Routes

PRADITA University

```
#lib/hiwi\_web/router.ex
defmodule HiwiWeb.Router do
  use HiwiWeb, :router

  scope "/auth", HiwiWeb do
    pipe_through :browser

    # Start OAuth login
    get "/:provider", AuthController, :request

    # GitHub callback endpoint
    get "/:provider/callback", AuthController, :callback
  end
end
```

```
/auth/github
/auth/github/callback
http://localhost:4000/auth/github
http://localhost:4000/auth/github/callback
```

Router Explanation

PRADITA University

- All Phoenix routes are defined in `lib/hiwi_web/router.ex`.
- The **/auth** scope groups all OAuth-related routes.
- `get "/:provider"` Starts the OAuth login flow (e.g., `/auth/github`).
- `get "/:provider/callback"` GitHub redirects here after user approval.
- Callback endpoint handles:
  - reading the authorization code,
  - exchanging it for an access token,
  - fetching user profile data,
  - creating a user session.
- Must match the callback URL registered in GitHub OAuth App.

Controller 1

```
# lib/hiwi_web/controllers/auth_controller.ex
defmodule HiwiWeb.AuthController do
  use HiwiWeb, :controller
  plug Ueberauth
  alias Hiwi.User
  alias Hiwi.Repo

  def callback(conn, params) do
    ...
  end

  defp signin(conn, changeset) do
    ...
  end

  defp insert_or_update_user(changeset) do
    ...
  end
end
```

PRADITA University

- **HiwiWeb.AuthController** handles GitHub OAuth login logic.
- **plug Ueberauth** injects Ueberauth into requests under `/auth/github` and `/auth/github/callback`.
- **callback/2** receives the GitHub response and parses token + user profile.
- **signin/2** stores the session, sets flash messages, and redirects.
- **insert_or_update_user/1** checks whether the user exists and inserts or reuses the record.
- These functions together implement the full GitHub login workflow.

```elixir
# lib/hiwi\_web/controllers/auth\_controller.ex
defmodule HiwiWeb.AuthController do
 ...
  def callback(conn, params) do
    IO.puts("conn = " <> Kernel.inspect(conn))
    IO.puts("conn: ")
    IO.inspect(conn)
    IO.puts("params = " <> Kernel.inspect(params))
    %{"code" => _code, "provider" => provider, "state" => _state} = params
    %{assigns: %{ueberauth_auth: auth}} = conn
    %{credentials: %{token: token}, info: %{email: email, nickname: nickname}} =
        auth

    user_params = %{
      token: token,
      email: email || nickname,
      provider: provider
    }

    IO.puts("user_params = " <> Kernel.inspect(user_params))

    changeset = User.changeset(%User{}, user_params)

    signin(conn, changeset)
  end
```

Controller 2 — Explanation

PRADITA University

- The **callback/2** function executes after GitHub redirects to:

```
/auth/github/callback
```

- Debug output printed:
  - full Plug.Conn
  - callback params
- OAuth data retrieved from:

```
conn.assigns.ueberauth_auth
```

- Extracted fields include:
  - access token
  - email / nickname

- The controller constructs:

```
user_params = %{
 token: token,
 email: email || nickname,
 provider: provider
}
```

- This is transformed into a changeset via:

```
User.changeset(%User{}, user_params
    )
```

- Finally, authentication flow continues:

```
signin(conn, changeset)
```

- This function inserts/loads the user and starts the session.

Controller 3

PRADITA University

```elixir
# lib/hiwi\_web/controllers/auth\_controller.ex
defmodule HiwiWeb.AuthController do
  ...
  defp signin(conn, changeset) do
    case insert_or_update_user(changeset) do
      {:ok, user} ->
        conn
        |> put_flash(:info, "Good to see you again!")
        |> put_session(:user_id, user.id)
        |> redirect(to: "/")

      {:error, reason} ->
        conn
        |> put_flash(:error, "Error when signing in: #{reason}")
        |> redirect(to: "/")
    end
  end
  ...
end
```

Controller 3 — Explanation

PRADITA University

Successful Login

- Calls:

```
insert_or_update_user(...)
```

- If user exists or is inserted:

```
put_flash(:info, ...)
put_session(:user_id, user.id)
redirect("/")
```

Error Handling

- If database fails:

```
{:error, reason}
```

- Shows error flash
- Redirects to home

Summary

- `signin/2` connects DB result flash redirect.
- Final step of OAuth login flow.

Controller 4

PRADITA University

```elixir
# lib/hiwi\_web/controllers/auth\_controller.ex
defmodule HiwiWeb.AuthController do
  ...
  defp insert_or_update_user(changeset) do
    case Repo.get_by(User, email: changeset.changes.email) do
      nil ->
        Repo.insert(changeset)

      user ->
        {:ok, user}
    end
  end
end
```

Controller 4 — Explanation

PRADITA University

Purpose

- Handles saving or loading a user during OAuth login.
- Prevents duplicate users by checking the email first.

Lookup Step

```
Repo.get_by(User,
  email: changeset.changes.email)
```

Behavior

- If no user exists:

```
Repo.insert(changeset)
```

- If user already exists:

```
{:ok, user}
```

- Ensures OAuth login always returns a valid user struct.

Summary

- Central logic for upsert-like user handling.
- Simplifies the login flow for both new and returning users.

Demo: Testing GitHub OAuth Login

PRADITA University

1. Start Phoenix Server

```
mix phx.server
```

2. Open App

```
http://localhost:4000
```

3. Click Login Button

```
OAuth GitHub
```

4. Redirect to GitHub

```
/auth/github
```

5. Authorize App

```
Authorize <username>
```

6. Callback to Phoenix

```
/auth/github/callback
```

7. Success Flash

```
Good to see you again!
```